

Thrashing

1. Thrashing (Simple English Explanation)

Thrashing happens when a process does not get enough memory (frames) to run properly. Because of this, the process keeps requesting pages that are not in memory, causing **page faults again and again**. The operating system keeps removing a page and bringing another one, but the removed page is needed immediately. So the system keeps swapping pages instead of doing real work.

When this continues, the system spends **more time handling page faults than actually executing programs**. This situation is called **thrashing**.

Real-world example:

Imagine a student studying from **5 different books**, but the table has space for **only 2 books**.

- The student keeps one book aside to place another on the table.
- After a minute, the removed book is needed again, so another book is removed.
- Most of the time is wasted in **changing books**, not studying.

2. Techniques to Handle Thrashing (Simple English + Real-World Examples)

i. Working Set Model

Simple explanation:

This technique is based on the idea of **locality**, which means a process uses a **small set of pages repeatedly** during a period of time.

The rule is simple:

- If a process is given **enough frames** to hold all the pages it is actively using (its *current locality*), page faults will be **very few**.
- Page faults will mostly happen **only when the process moves to a new part of the program** (new locality).
- But if the process is given **fewer frames than required**, it will keep replacing useful pages and will **start thrashing**.

Real-world example:

Think of a **cook preparing one dish**.

- For a dish, the cook needs **salt, oil, pan, and vegetables**.
- If all these are kept on the kitchen counter (enough space), cooking goes smoothly.

- If the counter is too small, the cook keeps putting things back and picking them again.

So, giving enough space for what is **currently needed** avoids unnecessary movement—just like giving enough frames avoids thrashing.

ii. Page Fault Frequency (PFF)

Simple explanation:

Thrashing happens when the **page fault rate becomes too high**.

So, instead of guessing memory needs, the OS **observes page faults** and reacts.

Rules:

- If page faults are **too frequent**, the process needs **more frames**.
- If page faults are **very rare**, the process probably has **more frames than needed**.
- The OS sets:
 - an **upper limit** (too many faults)
 - a **lower limit** (too few faults)

Based on this:

- If pf-rate > upper limit → **add a frame**
- If pf-rate < lower limit → **remove a frame**

By adjusting frames this way, thrashing can be **controlled and prevented**.

Real-world example:

Imagine traffic on a **highway lane**:

- If cars are getting stuck again and again (too many jams), **open a new lane**.
- If a lane is almost empty, **close it and use space elsewhere**.

Similarly, the OS adds or removes memory frames based on how often page faults occur, keeping the system running smoothly.

In short:

- **Working Set Model** ensures each process gets memory according to what it is actively using.
- **Page Fault Frequency** dynamically adjusts memory by watching page fault behavior.

Both techniques help the system **focus on real work instead of constant swapping**, preventing thrashing.